Table of Contents

1.Abstract.....2

2.Introduction.....2

3.Methodology.....3

4.Implementation.....6

5.Results & Analysis.....7

6.Discussion.....10

7.Conclusion.....11

PROJECT REPORT

ESCAPEROOMGAME2D: A 2D PUZZLE ESCAPE GAME USING JAVA

1. Abstract

This project develops EscapeRoomGame2D, a top-down 2D escape room puzzle game written in pure Java. The player is locked in connected rooms and must explore, collect items, solve puzzles and riddles, and reach the exit before a countdown timer ends. The methods implemented include a fixed-rate game loop, keyboard and mouse input, tile-based rooms loaded from text files, collision detection, an inventory, puzzle types (key and door, riddle with code keypad, lever sequence), progress tracking with room unlocking, Graphics2D drawing in Swing, sound effects, a score formula, and a built-in self-test. The system uses no external libraries or game engines. The results show that all rooms load correctly, collisions and puzzles behave as designed, and the player can win or lose through the complete game flow. The project demonstrates object-oriented design, event handling, file handling, and graphics programming in Java.

2. Introduction

-Background

Java is an object-oriented language known for platform independence and a strong standard library, including Swing and Java 2D for graphics. Escape room games need many cooperating parts: a game loop, input, rooms, items, rules, and a user interface. This makes them a good way to practise software design without any external game engine.

-Problem Statement

Students usually learn Java through small console programs, which rarely show how classes, inheritance, collections, and file handling work together in a larger interactive program. A graphical escape room game addresses this by applying object-oriented design to game state, input, rendering, level loading, and testing in one project that is easy to demonstrate.

-Objectives

- **To build a Java game with a stable game loop of about 60 updates per second.**
- **To create connected rooms and let the player move between them, with tile-based collision detection.**
- **To load room layouts from text files so new rooms need no code change.**
- **To implement logic puzzles, riddles, hidden clues, and interactive objects through an abstract Puzzle class.**

- **To implement items and an inventory that the player can collect and use to solve puzzles.**
- **To track progress, unlock new rooms after puzzles are solved, and provide game-over and victory conditions.**
- **To design menus, a HUD, sound effects, and visual feedback.**
- **To verify the game logic with a built-in automated self-test (SelfTest).**

-Scope

The project is a desktop game in pure Java (JDK 11+) with three connected rooms, a countdown timer, and win and lose screens. It is single-player and local. It does not include multiplayer, a level editor, 3D graphics, or database storage.

3. Methodology

-Approach

The project follows a modular, object-oriented approach built around the game cycle: read input, update the game state, and render the screen. The game state (running, paused, won, lost) and the progress of the player (rooms unlocked, items collected, puzzles solved) are held in GameState, each level in a Room object, and the player in a Player object, so game rules stay separate from drawing code. The game is deterministic: the same actions always give the same result.

Mapping from game design to code:

Game Element	Implemented As	Details
Room	Room object	Tile grid loaded from a .txt file; 3 rooms (Cell, Library, Control Room) with a short description
Wall / floor / door	Tile characters	# wall, . floor, D door, S start; walls and locked doors block the player
Player	Player class	Arrow keys or WASD; hit box smaller than a tile; moves between rooms through doors
Item	Item class	Key, note, battery, tool; picked up with E; some are hidden clues
Inventory	Inventory class	Up to 6 items shown in the HUD; items can be used on objects
Locked door	LockedDoor	Opens when the matching key is in the inventory
Riddle + keypad	CodeLock	A riddle note gives the 4-digit code; wrong code costs score
Lever puzzle	LeverPuzzle	Levers in the correct order; wrong order resets them

Progression	**GameState**	**Next room unlocks after its puzzle is solved; exit gives victory**
Time limit	**Countdown timer**	**10 minutes; game is lost at 0**
Hint	**Hint system**	**Press H; each hint costs 30 seconds**

Scores used by the system:

Score	Formula
Time left	**timeLimit − timeUsed (seconds)**
Final score	**max(0, 1000 + timeLeft × 2 − hintsUsed × 50 − wrongCodes × 25)**
Rank	**≥ 1500 Master Escapist; 1000–1499 Detective; < 1000 Rookie**

-Algorithm

1. **Start Main, create the window (JFrame) and GamePanel, and register keyboard and mouse listeners.**
2. **Load room files (room1.txt to room3.txt) with Files.readAllLines into tile arrays; place items, puzzles, and the player.**
3. **Start the game loop (about 60 updates per second); repeat steps 4 to 8.**
4. **Read input: pressed keys go into a set; mouse clicks go to the active puzzle.**
5. **Update the player: CollisionChecker accepts a new position only if it is free of walls, locked doors, and solid objects.**
6. **Check interactions: pressing E near an item or puzzle picks it up or starts it; puzzles check their own rules, and a solved puzzle unlocks the next room.**
7. **Update the timer and state: reaching the exit gives WON; time at 0 gives LOST.**
8. **Render the frame with Graphics2D: floor, walls, objects, player, HUD, and win or lose screen.**
9. **Show final score and rank; R restarts, Esc quits.**

-Flowchart

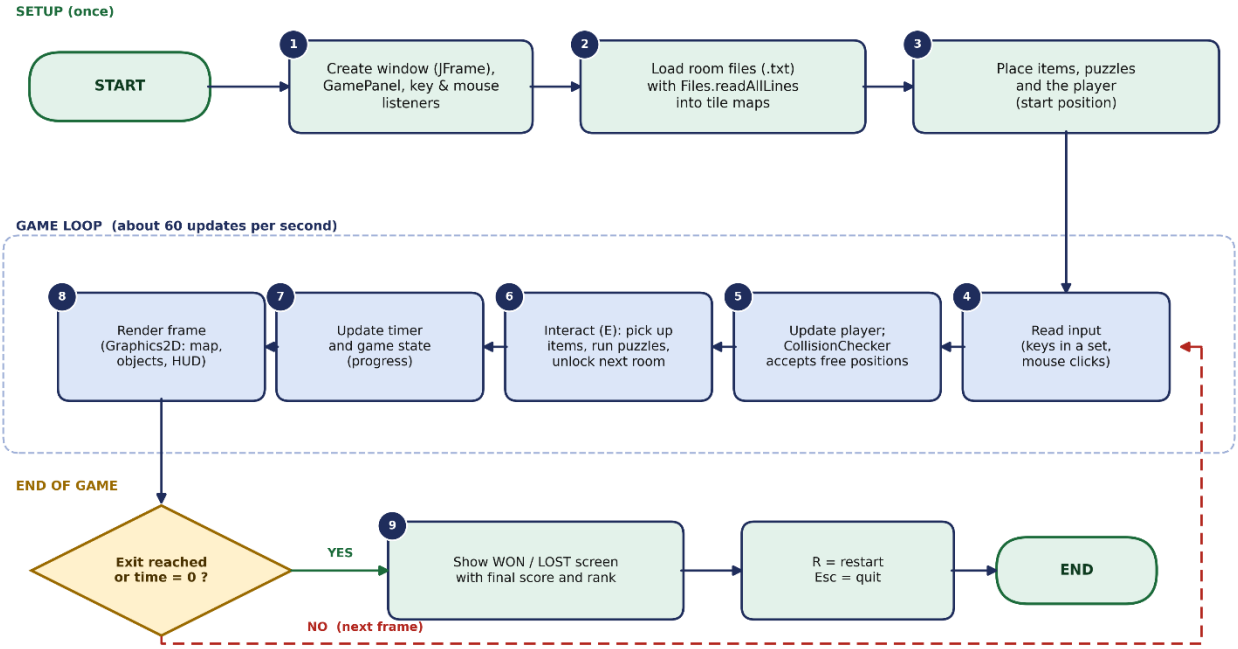


Fig 1: Flowchart of ESCAPEROOMGAME2D

-Tools and Technologies

The game is built with Java JDK 11+ in VS Code, using javax.swing and java.awt.Graphics2D for the window and drawing, java.awt.event for keyboard and mouse input, java.nio.file for reading room files, javax.sound.sampled for sound effects, and java.util collections for storing rooms, items, and keys. The Java and Swing documentation were used as reference. The program was developed and tested with JDK **[version]** and needs only a JDK (11 or newer). Hardware: Laptop with keyboard and mouse.

-Experimental Design

The system is verified by playing all three rooms from start to finish and with the SelfTest class, which needs no test library and exits with code 0 when every check passes. SelfTest contains **[XX]** checks in four groups:

- **Map loading:** every room loads; the start is on a floor tile; missing or empty files do not crash the game.
- **Movement and collision:** the player cannot enter walls or locked doors.
- **Items and puzzles:** items are picked up once; doors need the right key; the keypad accepts only the right code; the lever puzzle resets on a wrong order.

- **Game flow:** the timer works; rooms unlock after puzzles; WON and LOST states are reached correctly; the score is never negative.

4. Implementation

-System Architecture

1. **Game Core Module:** Main, GameWindow, and GamePanel run the window, game loop, input, and game state.
2. **Game Logic Module:** Player, Room, RoomLoader, CollisionChecker, Item, Inventory, and the Puzzle classes hold the rules and data.
3. **Presentation Module:** HUD, menus, the paint methods of GamePanel, and SoundPlayer draw the screen and play sounds.

-Components

- **Main.java:** Entry point; creates the window and starts the loop.
- **GamePanel.java:** Runs the loop, receives input, updates, and paints each frame.
- **Player.java:** Position, speed, direction, and movement.
- **Room.java / RoomLoader.java:** Room data and loading from text files.
- **CollisionChecker.java:** Checks overlap with walls, locked doors, and solid objects.
- **Item.java / Inventory.java:** Collectable objects and the player's item list (up to 6 items).
- **Puzzle.java, LockedDoor.java, CodeLock.java, LeverPuzzle.java:** Puzzles sharing the same methods (interact, isSolved).
- **HUD.java:** Timer, inventory, hints, and end screens.
- **SoundPlayer.java:** Short sound effects.
- **SelfTest.java:** Automated checks.

-Weekly Workflow

Week	Topic	Result
1	Project Planning and Research	Game concept and storyline defined; escape room games and puzzle mechanics researched; theme and objectives finalized
2	Requirement Analysis	Game features, player objectives, and win conditions identified; overall architecture designed

3	Project Setup	Java environment set up; project structure and classes created; game assets and resources prepared
4	Room Design and Navigation	Rooms and locations created; player movement between rooms; room descriptions and interactions
5	Puzzle Implementation	Logic puzzles and riddles; hidden clues and interactive objects; puzzle-solving mechanisms
6	Inventory System	Item collection; inventory management; item usage for solving puzzles
7	Game Logic and Progression	Progress tracking; rooms unlock after puzzles; game-over and victory conditions
8	User Interface Enhancement	Menus and game screens; improved interaction; sound effects and visual feedback
9	Integration and Testing	All modules combined; puzzles and game flow tested; bugs fixed and performance improved
10	Finalization and Documentation	Gameplay optimization, final bug fixing, README, project documentation, final testing, and submission-ready project

5. Results & Analysis

-Result

The application opens a game window and lets the player complete all three rooms. All **[XX]** self-test checks passed. Room 1 (Cell) was completed in about [1 min], Room 2 (Library) in about [3 min], and Room 3 (Control Room) in about [3 min]. A full game within 10 minutes ended with the WON screen and a score of [value], and letting the timer reach 0 correctly showed the LOST screen. The game ran at a stable [60] frames per second on the test laptop, and a wrong keypad code reduced the score by 25 points as designed.

-Analysis

A simple loop of input, update, and render, with tile-based maps and rule-based puzzle classes, is enough to build a complete 2D puzzle game in pure Java. The abstract Puzzle class made new puzzle types easy to add, and text-file rooms made level design fast. Because game rules do not depend on drawing code, the logic could be tested without opening a window. The structured 10-week plan, which built rooms, puzzles, inventory, and progression one after another, kept each module small and testable before integration.

-Code (Output)



Figure 2: Mein Menu



Figure 3: Room 1 (Cell) with player and HUD

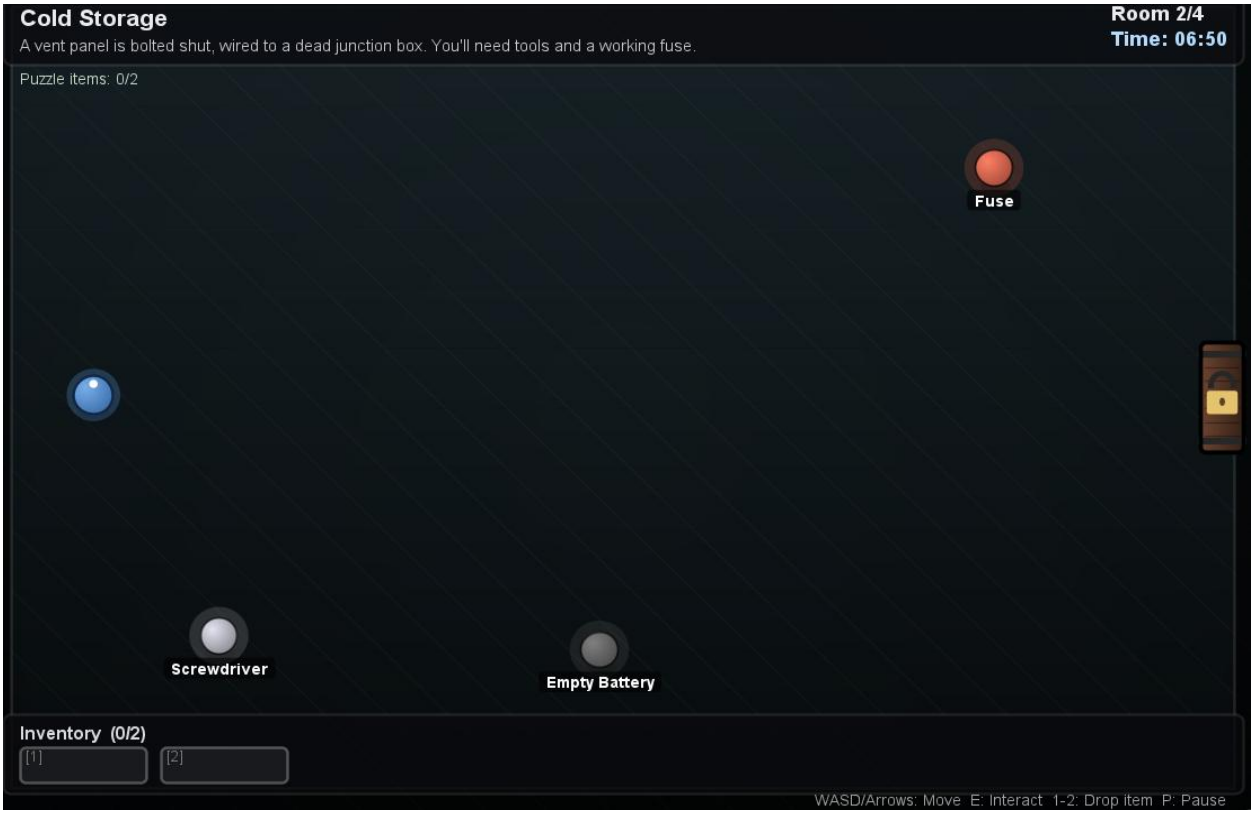


Figure 4: Keypad puzzle in Room 2 (Library)

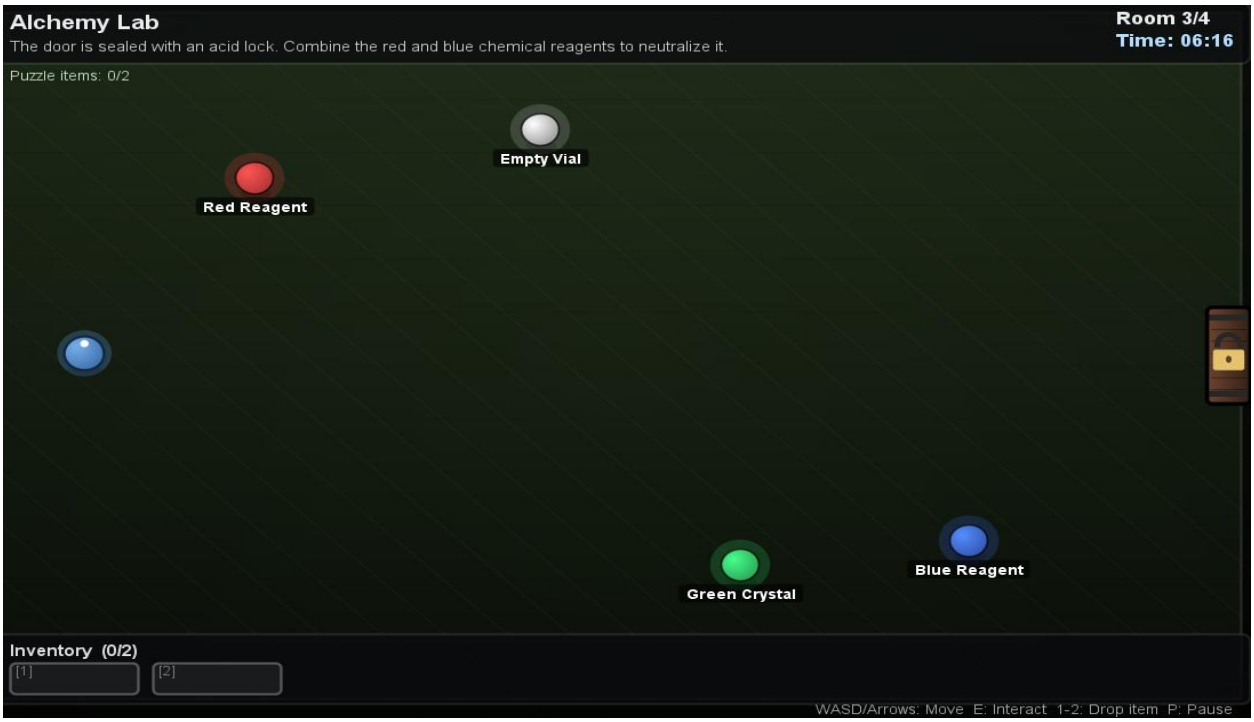


Figure 5: Win screen with score and rank



Figure 6: SelfTest result (all checks passed)

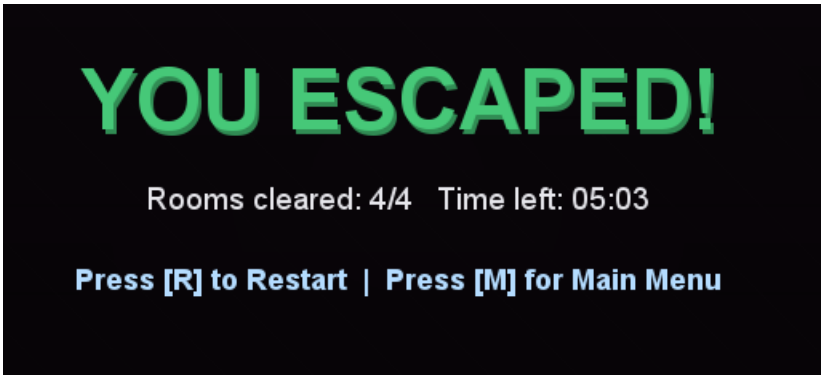


Figure 7:Winner

6. Discussion

-Interpretation

Pure Java can run a real-time game loop, handle input, load levels from files, apply puzzle rules, draw 2D graphics, and test its own logic without external libraries. Splitting the work into core, logic, and presentation modules keeps the code simple and maintainable. The system meets its goals: a playable game with three rooms, several puzzle types, progression, a working timer and score, and a clear win and lose flow.

-Key Technical Achievements

- **Modular Architecture:** Rules, data, and drawing are separate.
- **Data-driven Levels:** New rooms need no new Java code.
- **Polymorphic Puzzles:** All puzzles share one base class.
- **Zero-Dependency Design:** Only a JDK is needed.
- **Built-in Verification:** SelfTest checks maps, collision, puzzles, and game flow.

-Challenges

- **Stable Game Loop:** Speed differed between computers; solved with a fixed time step using System.nanoTime().
- **Collision at Corners:** The player got stuck on corners; solved by testing X and Y movement separately.
- **Puzzle State:** Solved puzzles had to stay solved; state is stored in the Room and Puzzle objects.
- **Game Balance:** The 10-minute limit was set after play-testing.

-Limitations & Potential Enhancements

1. Fixed Content: Only three rooms and a few puzzle types.
2. No Save Function: Progress cannot be saved.
3. Simple Graphics: Shapes only; sprites would improve the look.
4. Further Ideas: a level editor, random codes, a high score table, and multiplayer.

7. Conclusion

-Summary

EscapeRoomGame2D shows how foundational Java concepts (object-oriented design, inheritance, polymorphism, collections, event handling, file handling, and Graphics2D) can be combined into a complete 2D game. Following the 10-week plan, the project moved from planning and requirement analysis (Weeks 1–2) and setup (Week 3) to rooms and puzzles (Weeks 4–5), inventory and game logic (Weeks 6–7), user interface enhancement (Week 8), integration and testing (Week 9), and finalization with gameplay optimization, README, documentation, and final testing (Week 10). The final game lets the player explore three rooms, solve puzzles, and escape against the clock, and passes all of its own checks.

-Contributions

The project builds a complete interactive game with native Java libraries only. Key contributions include a stable game loop, a data-driven room loader, a collision system.